

GitHub OAuth2 — Implémentation dans SUPMEAL

Evan Morvant

Ce document explique comment l'authentification GitHub est implémentée et comment la configurer en local. Le flux est **identique à celui de Google** — mêmes endpoints, même `state` signé, même redirection finale ; seuls la console du fournisseur et le traitement de l'adresse e-mail changent. Le guide Google reste la référence pour le détail du mécanisme : `oauth-google.md`.

Aperçu

- Libs : `passport`, `passport-github2`, `jsonwebtoken`.
- Endpoints backend :
 - `GET /api/v1/auth/oauth/github` — démarre le flux.
 - `GET /api/v1/auth/oauth/github/callback` — échange le `code`, résout l'utilisateur, redirige vers le client avec les jetons dans le fragment.
 - `POST /api/v1/users/me/oauth/github` — rattache GitHub à un profil déjà connecté.
- Portée demandée : `user:email`. Sans elle, GitHub ne renvoie aucune adresse — voir plus bas.

Prérequis

- Un compte GitHub.
- Documentation officielle : <https://docs.github.com/en/apps/oauth-apps/building-oauth-apps>

L'URL de callback à déclarer est celle de **l'API** :

```
http://localhost:4000/api/v1/auth/oauth/github/callback
```

Mise en place pas-à-pas

1. Créer l'application OAuth

- GitHub → *Settings* → *Developer settings* → *OAuth Apps* → **New OAuth App**.
Lien direct : <https://github.com/settings/developers>

2. Renseigner le formulaire

Champ	Valeur
Application name	SUPMEAL
Homepage URL	<code>http://localhost:8080</code>
Authorization callback URL	<code>http://localhost:4000/api/v1/auth/oauth/github/callback</code>

Une application OAuth GitHub n'accepte **qu'une seule** URL de callback. Pour travailler à la fois en développement (API sur 4000) et sur un serveur déployé, il faut créer deux applications distinctes.

3. Récupérer les identifiants

- Le **Client ID** s'affiche immédiatement.
- Le **Client Secret** se crée avec *Generate a new client secret*, et **ne sera plus jamais affiché** ensuite : le copier tout de suite.

4. Renseigner les variables d'environnement

Dans le `.env` à la racine du dépôt :

```
GITHUB_CLIENT_ID=<client_id>
GITHUB_CLIENT_SECRET=<client_secret>
API_PUBLIC_URL=http://localhost:4000
CLIENT_ORIGIN=http://localhost:8080
```

L'URL de callback envoyée à GitHub est construite à partir d' `API_PUBLIC_URL` ; la redirection finale vers l'application, à partir de `CLIENT_ORIGIN`.

5. Relancer les services

```
docker compose up -d api
```

Aucune reconstruction d'image n'est nécessaire : seules les variables changent, et Compose recrée le conteneur pour les injecter.

La stratégie GitHub n'est enregistrée qu'au démarrage, et seulement si les deux variables sont renseignées.

6. Tester

- <http://localhost:8080/register> → « S'inscrire avec GitHub » → autorisation → retour automatique sur l'application, connecté.
- Depuis *Paramètres* → *Compte*, le bouton **Lier** rattache GitHub à un profil existant.

La particularité GitHub : l'adresse e-mail

Google renvoie toujours l'adresse du compte. GitHub, non — c'est la seule vraie différence de traitement entre les deux fournisseurs, et elle a des conséquences visibles :

- Si l'utilisateur a **masqué son adresse** dans ses réglages de confidentialité GitHub, le profil renvoyé n'en contient aucune. SUPMEAL crée alors le compte avec une adresse technique de la forme `github_<id>@oauth.local`. Le compte fonctionne, mais cette adresse ne reçoit rien et n'est pas modifiable depuis l'application : elle sert d'identifiant, pas de moyen de contact.
- **Conséquence sur les invitations** : une invitation à un cookbook se fait par adresse e-mail. Un compte GitHub sans adresse réelle ne peut donc pas être invité sous son adresse GitHub. Le contournement est de créer le compte par e-mail et mot de passe, puis de **lier** GitHub depuis les paramètres.
- Quand l'adresse **est** renvoyée et qu'elle correspond à un compte SUPMEAL existant, le compte GitHub est rattaché à ce profil plutôt que d'en créer un second. C'est le comportement voulu : une personne, un compte.

La portée `user:email` est déjà demandée par la stratégie ; elle donne accès aux adresses, y compris non publiques, à condition que l'utilisateur accepte la demande d'autorisation.

Si ça ne marche pas

Symptôme	Cause probable
503 PROVIDER_NOT_CONFIGURED	GITHUB_CLIENT_ID ou GITHUB_CLIENT_SECRET vide, ou API non redémarrée depuis la modification du .env .
The redirect_uri MUST match the registered callback URL	L'URL déclarée dans l'application GitHub diffère de API_PUBLIC_URL + /api/v1/auth/oauth/github/callback , au caractère près (http ≠ https , port compris).
Retour sur /oauth/callback#error=OAUTH_ACCOUNT_TAKEN	Ce compte GitHub est déjà rattaché à un autre profil SUPMEAL. Le délier depuis ce profil avant de le relier.
Retour sur /oauth/callback#error=state_invalide	Plus de 10 minutes entre le départ et le retour, ou secrets JWT changés entre-temps.
Compte créé avec une adresse en @oauth.local	Adresse masquée côté GitHub, ou autorisation user:email refusée. Voir la section ci-dessus.
Nouveau compte créé alors qu'un profil existait	L'adresse GitHub n'est pas celle du compte SUPMEAL. Délier, puis relier depuis Paramètres → Compte du bon profil.

Documentation réalisée par Evan Morvant.